МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №4
по курсу «Технологии параллельного программирования»

Работа с матрицами. Метод Гаусса

Выполнил: Немкова А. Р.
Группа: М8О-408Б-22
Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2025

Условие

Цель работы. Использование объединения запросов к глобальной памяти. Реализация метода Гаусса с выбором главного элемента по столбцу. Ознакомление с библиотекой алгоритмов для параллельных расчетов Thrust. Использование двухмерной сетки потоков. Исследование производительности программы с помощью утилиты nvprof.

Вариант №2. Вычисление обратной матрицы.

Входные данные. На первой строке задано число n – размер матрицы. В следующих n строках, записано по n вещественных чисел – элементы матрицы. ($n \leq 10^4$)

Выходные данные. Необходимо вывести на n строках, по n чисел – элементы обратной матрицы.

Программное и аппаратное обеспечение

Графический процессор:

- Модель: NVIDIA GeForce RTX 3050 Laptop GPU
- Compute Capability: 8.6
- Объем графической памяти: 4 294 443 008
- Разделяемая память на блок: 49 152
- Количество регистров на блок: 65 536
- Размер warp: 32
- Максимальное количество потоков на блок: (1024 × 1024 × 64)
- Максимальное количество блоков: (2 147 483 647 × 65 535 × 65 535)
- Объем константной памяти: 65 536
- Количество мультипроцессоров (SM): 16

Центральный процессор:

- Модель: AMD Ryzen 7 5800H with Radeon Graphics 3.20 GHz
- Количество ядер: 8
- Количество потоков: 16
- Базовая тактовая частота: 3,2 ГГц
- Максимальная тактовая частота: 4,4 ГГц
- Кеш L1: 64 Кб (на ядро)
- Кеш-память L2: 512 КБ (на ядро)
- Кеш L3: 16 Мб

Оперативная память:

- Тип памяти: DDR4
- Объём памяти: 2 * 8 Гб
- Частота: 3200 Мт/с

Жёсткий диск:

- Тип: SSD
- Модель: SK Hynix HFM001TD3JX013N
- Объём памяти: 1 Тб
- Скорость записи/чтения: 3100/3500 МБ/с

Программное обеспечение:

- OS: Ubuntu 20.04 Linux (WSL2)
- IDE: VS Code

- Compiler: nvcc, gcc

Метод решения

Основная идея алгоритма заключается в преобразовании расширенной матрицы [A|E], где E - единичная матрица того же размера, к виду [E|A^{-1}] с помощью элементарных операций над строками. Этот процесс состоит из двух основных этапов: прямого и обратного хода.

Прямой ход метода Гаусса направлен на приведение матрицы A к верхней треугольной форме. На каждом шаге выполняются следующие операции:

1. Выбор главного элемента. Для обеспечения численной устойчивости осуществляется поиск максимального по модулю элемента в i-м столбце, начиная с i-й строки:

$$p = argmax_{k=i}^{n-1} |a_{ki}|$$

Это позволяет избежать деления на малые числа и уменьшить погрешность вычислений.

2. Перестановка строк. Если максимальный элемент находится не в i-й строке, производится перестановка строк i и p. Такая же перестановка выполняется и для соответствующей части единичной матрицы.
3. Нормировка i-й строки: Ведущий элемент приводится к единице путем деления всей строки на элемент a_{ii}.
4. Исключение элементов в i-м столбце. Из всех последующих строк вычитается i-я строка, умноженная на соответствующий коэффициент:

$$R_j \leftarrow R_j - a_{ji} * R_i$$

Обратный ход начинается после завершения прямого хода и преобразует верхнюю треугольную матрицу в единичную. На этом этапе для каждого i от n - 1 до 0 выполняются аналогичные операции исключения, но уже для элементов выше главной диагонали.

Описание программы

Программа реализована в виде одного файла, содержащего 5 функций и 2 макроса.

Основные типы данных:

- double - основной вещественный тип для хранения элементов матриц и выполнения вычислений
- Структура SCompare - функтор для сравнения абсолютных значений чисел, используемый библиотекой Thrust при поиске максимального элемента

Программа использует несколько макросов и констант:

- Макрос CSC - обеспечивает автоматическую проверку ошибок CUDA с выводом диагностической информации о файле и строке, где произошла ошибка
- BLOCK_SIZE и GRID_SIZE - определяют размеры блоков (32) и сетки (64) для организации параллельных вычислений на GPU
- EPS - задает значение epsilon (10^{-10}) для проверки вырожденности матрицы и обеспечения требуемой точности вычислений.

Реализованные ядра CUDA:

- Ядро перестановки строк swap_row_kernel. Данное ядро выполняет параллельную перестановку двух строк в матрицах. Каждый поток обрабатывает определенный столбец. Ядро работает одновременно с

исходной матрицей и расширенной частью, выполняя идентичные операции для обеих.

- Ядро нормировки строки normalize_row_kernel. Это ядро осуществляет нормировку строки матрицы путем деления всех ее элементов на ведущий элемент. Операция выполняется параллельно для всех столбцов.
- Ядро прямого хода forward_kernel. Ядро реализует этап прямого хода метода Гаусса, используя двумерную организацию потоков. Для каждой строки ниже текущей ведущей строки вычисляется множитель и выполняется исключение соответствующих элементов в исходной матрице и единичной матрице.
- Ядро обратного хода backward_kernel. Аналогично прямому ходу, это ядро выполняет операции обратного хода метода Гаусса, но работает с элементами выше главной диагонали. Оно обеспечивает приведение расширенной матрицы к полностью единичному виду, завершая процесс вычисления обратной матрицы. Ядро обрабатывает элементы единичной матрицы, выполняя обратное исключение.

Главная функция main() организует последовательное выполнение алгоритма: чтение входных данных, инициализацию матриц, передачу данных на GPU, выполнение прямого и обратного хода метода Гаусса, и вывод результата.

Библиотека Thrust используется для поиска максимального по модулю элемента в столбце матрицы на каждом шаге алгоритма с помощью специального функтора сравнения.

Результаты

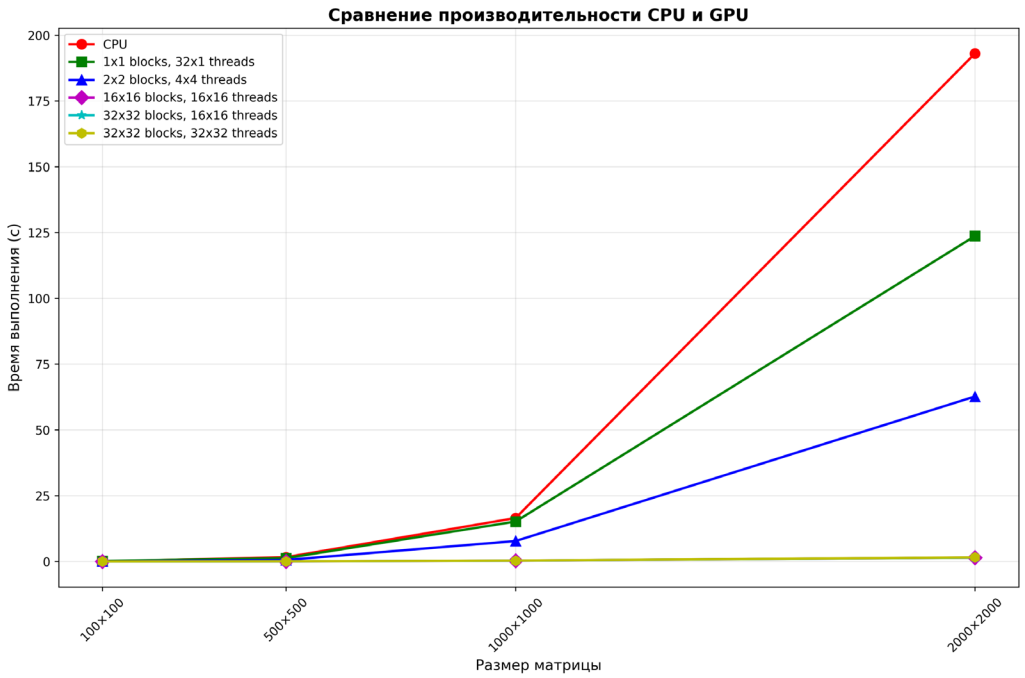
Были проведены тесты производительности GPU ядра с различными конфигурациями запуска для матриц размеров 10×10, 100×100, 500×500, 1000×1000 и 2000×2000.

Конфигурация	100×100	500×500	1000×1000	2000×2000
<<dim3(1, 1), dim3(32, 1)>>	0.230403 s	1.293774 s	15.195855 s	123.640240 s
<<dim3(2, 2), dim3(4, 4)>>	0.028625 s	0.593826 s	7.787829 s	62.698446 s
<<dim3(16, 16), dim3(16, 16)>>	0.015518 s	0.053335 s	0.334169 s	1.525872 s
<<dim3(32, 32), dim3(16, 16)>>	0.014479 s	0.056640 s	0.334574 s	1.519339 s
<<dim3(32, 32), dim3(32, 32)>>	0.021783 s	0.075349 s	0.358542 s	1.581628 s

Также были выполнены замеры работы с использованием CPU

Размер матрицы	Время работы
100×100	0.004885 s
500×500	1.660102 s
1000×1000	16.529674 s
2000×2000	192.983693 s

Ниже представлен график сравнения замеров времени работы GPU с конфигурациями <<dim3(1, 1), dim3(32, 1)>>, <<dim3(2, 2), dim3(4, 4)>>, <<dim3(16, 16), dim3(16, 16)>>, <<dim3(32, 32), dim3(16, 16)>>, <<dim3(32, 32), dim3(32, 32)>> и CPU на тестах с размерами матриц 100×100, 500×500, 1000×1000 и 2000×2000.



Было проведено профилирование алгоритма с использованием утилиты nvprof для матрицы размером 500×500 с конфигурацией запуска <<dim3(32, 32), dim3(64, 64)>>. Основные вычислительные ядра заняли 91.85% общего времени выполнения:

- forward_kernel (прямой ход метода Гаусса) - 55.16% времени, 113.32 мс
- backward_kernel (обратный ход метода Гаусса) - 36.69% времени, 75.37 мс

Вспомогательные операции показали высокую эффективность:

- normalize_row_kernel (нормировка строк) - 3.49% времени, 7.18 мс
- Операции поиска максимального элемента с использованием Thrust - всего 1.79% времени

Анализ вызовов API CUDA выявил, что основную часть времени (55.98%) занимали операции синхронизации cudaDeviceSynchronize, что указывает на необходимость оптимизации управления параллельным выполнением. Операции работы с памятью показали хорошую эффективность - копирование данных между CPU и GPU заняло менее 2% общего времени.

Выводы

В ходе данной лабораторной работы была реализована параллельная версия метода Гаусса с выбором главного элемента по столбцу для вычисления обратной матрицы с использованием технологии CUDA.

Сравнение производительности показало, что GPU реализация обеспечивает ускорение до 127 раз для матрицы 2000×2000 по сравнению с CPU версией. Наилучшие результаты достигнуты при конфигурации 32×32 блоков по 16×16 потоков. Меньшие конфигурации демонстрируют низкую производительность из-за недостаточного использования параллельных возможностей GPU, а чрезмерно большие конфигурации приводят к росту накладных расходов.

Профилирование выявило, что 91.85% времени выполнения занимают основные вычислительные ядра (прямой и обратный ход). Операции синхронизации являются основным ограничением производительности, что характерно для итеративных алгоритмов с последовательной зависимостью шагов.

Данный алгоритм находит применение в различных областях вычислительной математики и инженерии. Он используется при решении систем линейных уравнений в задачах машинного обучения, компьютерного зрения и физического моделирования. В инженерии метод применяется для анализа электрических цепей, расчета конструкций и обработки сигналов.